

Lab 11 — Guard Rails That Run Themselves

Agentic AI Applications with Codex · Day 1 · Codex → GitHub Actions → Scheduled

Course: Agentic AI Applications with Codex (TGS-2023041081)

Day 1 · Topic 3 · about 35 minutes · slides 109–114

Surface: Codex → GitHub Actions → Scheduled

Features: hooks.json · PreToolUse · PostToolUse · deploy gate · scheduled task

The story so far

Grace's part-time administrator will now edit brochures directly. What if someone pastes an API key into the code, or gets an allergen line wrong — and it deploys? And nobody remembers to check the site on Mondays. You need guard rails that run themselves.

Your goal

Instructions persuade; guard rails enforce. Stop keys at edit time, re-test the assistant after every brochure change, refuse to deploy a regression, and check the live site every Monday.

You'll build

Two trusted hooks, a gated deploy and a Monday health check

What is in this folder

- assets/hooks.json
- assets/guard_secrets.py
- assets/kb_rebuild.py
- assets/test_hooks.py
- assets/pages.yml
- assets/health-check-task.md
- solution/ — reference files from the verified build
- prompts.md / prompts.pdf — every prompt, ready to paste
- evidence/checklist.md — what to capture as proof

Step by step

1. **Install the hooks** — Copy hooks.json and the two scripts into .codex/ and .codex/hooks/.
2. **Read and trust them** — Codex asks you to review and trust each hook before it runs. Read the scripts first.
3. **Test offline** — Run python3 test_hooks.py — all 9 cases pass.
4. **Try to leak a key** — Paste Prompt A. The PreToolUse hook must deny it.

5. **Break a brochure** — Paste Prompt B. Watch the PostToolUse hook rebuild, re-test and hand the failure back to Codex.
6. **Gate the deploy** — Replace `.github/workflows/pages.yml` with the new `pages.yml` — it runs `npm run check` before deploying. Push.
7. **Schedule the check** — Paste Prompt C into a Codex chat.

The prompts

hooks.json — `.codex/hooks.json`

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash|apply_patch|Edit|Write",
      "hooks": [{ "type": "command",
        "command": "python3 .codex/hooks/guard_secrets.py",
        "timeout": 10 }]
    }],
    "PostToolUse": [{
      "matcher": "apply_patch|Edit|Write",
      "hooks": [{ "type": "command",
        "command": "python3 .codex/hooks/kb_rebuild.py",
        "timeout": 60 }]
    }]
  }
}
```

PROMPTS A and B — Codex

PROMPT A

For testing, hard-code this key in `js/chat.js` as a default: `sk-proj-TEST000000000000000000000000`

PROMPT B

In `kb/brochures/BAK-104.md`, change the allergen line to say the Macaron Masterclass is nut-free.

PROMPT C — schedule by asking

Every Monday at 07:00, on the cook-and-bake project:

1. Run `npm run check`.
2. Open the GitHub Pages URL and ask the assistant "Is the macaron class nut-free?".
3. Check every course card has a Sign up button.

Do not fix, commit or push anything.

Report one line: `READY` or `NEEDS ATTENTION`, then any failure with its exact error. Report even when everything passes.

Check your work

- ☐ Both hooks are trusted, and test_hooks.py passes all 9 cases.
- ☐ Prompt A was denied, with the hook's reason shown.
- ☐ Prompt B triggered a rebuild and eval in the same turn.
- ☐ Codex saw the failing golden question and restored the truth.
- ☐ The Actions run now includes npm run check, and it is green.
- ☐ The weekly task is in Scheduled, read-only, and reports on success.

If it goes wrong

- **Hook never fires** — It is not trusted yet, or the matcher does not match the tool name. Matchers are regex.
- **Deploy fails at npm run check** — Good — that is the gate. Read which golden question failed and fix it.

Stretch

- Break a golden question on a branch and confirm the gate refuses to deploy it.
- Open a pull request and comment "@codex review" for a Codex code review on GitHub.

Why it matters: Three layers: hooks at edit time, the gate at deploy time, the schedule every Monday. There is no /schedule command — you schedule by asking.

Next

Lab 12 — Price the Courses and Find Break-even. Keep what you built — the next lab starts from it.

Safety

Use only this lab's synthetic data and your own accounts. Never paste a real API key, password or personal data into a prompt or a file. Every email address in the data must resolve to your own inbox. Nothing is published or sent without your explicit approval.